J Shanghai Univ (Engl Ed), 2011, 15(1): 63–68
Digital Object Identifier(DOI): 10.1007/s11741-011-0693-2

Construction for M-arrays and application in structured light

JIA Xiao-jun (贾小军)[1,2], ZHANG Zhi-jiang (张之江)[1], YU Qing-cang (喻擎苍)[3]
1. Key Laboratory of Specialty Fiber Optics and Optical Access Networks, School of Communication and Information Engineering, Shanghai University, Shanghai 200072, P. R. China
2. College of Mathematics, Physics and Information Engineering, Jiaxing University, Jiaxing 314001, Zhejiang, P. R. China
3. School of Informatics and Electronics, Zhejiang Sci-Tech University, Hangzhou 310018, P. R. China

©Shanghai University and Springer-Verlag Berlin Heidelberg 2011

Abstract M-arrays are random arrays in which an appropriate sub-window appears only once in the whole array. Coded structured light based on M-arrays is one-shot technique to rapidly acquire 3D information of unknown surfaces by projecting suitable patterns onto a measuring surface. This paper presents a method to construct large size M-arrays based on the piece growing algorithm in which an array is constructed by many pieces through splicing each other. Reconstructing 3D shapes by utilizing the designed pattern based on constructed M-arrays for two objects are given.

Keywords M-arrays, structured light, piece growing, 3D reconstruction

Introduction

Structured light technique is considered as one of the most reliable techniques to reconstruct 3D shape of objects. Generally, it is constituted by a light source device and a camera, also known as coded structured light[1]. Recently, structured light techniques have been widely used in many fields, such as 3D reconstruction, industrial inspection, object recognition, reverse engineering, biometrics and others[2–3].

A proper illumination pattern is critical for coded structured light technique. Salvi, *et al.*[1,4] have given well summarizations on these techniques. Among them, one-shot techniques with M-arrays (perfect maps) or pseudorandom arrays patterns are robust against occlusions, and are the best suited for dynamic scenes. M-arrays first presented by Etzion[5] are pseudorandom arrays of dimensions $n_1 \times n_2$ in which a sub-matrix of dimensions $k_1 \times k_2$ appears only once in the whole pattern. Perfect maps are constructed theoretically with dimensions $n_1 \times n_2 = q^{k_1 k_2}$, but for real applications the zero sub-matrixes are not considered. Macwiliams, *et al.*[6] proposed constructions for similar arrays with size $n_1 \times n_2 = q^{k_1 k_2} - 1$ such that each nonzero submatrix appears exactly once as a window in the array, called pseudorandom arrays. Most M-arrays and pseudorandom arrays can be constructed by the diagonal algorithm based on De Bruijn sequence and pseudorandom sequence, respectively.

Choosing an appropriate window property will determine the robustness of the pattern against pattern occlusions and object shadows for a given application. Morita, *et al.*[7] proposed an approach to reconstruct 3D surfaces of objects by projecting a binary M-array pattern with 32×27. Kiyasu, *et al.*[8] presented a method to measure 3D shape of specula polyhedrons by utilizing an M-array with 18×18 circular spots. Some similar patterns based on arrays are presented to project[9–11].

However, if a 1 024×768 pixels projector plane is spread with code words among which any code word occupies 7×7 pixels and is spaced by 1 pixel, there will then be 128×96=12 288 code words on the plane. 14 code bits and a minimum window property of 5×3 are required to generate such a large number of code words while utilizing monochrome illumination. With the increasing of the window size, it is getting more complex to seek neighbors in heavily distorted images.

In order to solve this problem, a large size array should be constructed. An algorithm called piece growing is presented in this paper to construct the large size M-arrays. Choosing appropriate pieces (sub-windows)

Received Sept.10, 2010; Revised Nov.25, 2010
Project supported by the National Natural Science Foundation of China (Grant No.60772124), the State Key Program of National Natural Science of China (Grant No.60832003), the Shanghai Leading Academic Discipline Project (Grant No.S30108), and the Foundation of the Science and Technology Commission of Shanghai Municipality (Grant No.08DZ2231100)
Corresponding author ZHANG Zhi-jiang, Ph D, Prof, E-mail: zjzhang@shu.edu.cn

to splice into the array can construct the whole array. According to the algorithm, 10-ary 4 degree M-arrays with dimensions of 79×59 can be constructed. A pattern with resolutions of 1 024×768 pixels spread by utilizing 10 symbols based on the M-array is proposed. In addition, the results of reconstruction for two objects are given.

1 De Bruijn and pseudorandom sequences

1.1 De Bruijn sequence and its construction

$A = (a_0, a_1, a_2, \cdots)$ is an infinite periodic sequence, where $a_i \in Q, Q = \{0, 1, \cdots, q-1\}$, and q is a given natural number $q \geqslant 1$. Suppose that the cycle of A is $p(A) = q^n$, where n is a given natural number $n \geqslant 1$ and $A_i = (a_i, a_{i+1}, \cdots, a_{i+n-1}), i = 0, 1, \cdots, q^n - 1$, which consists of all possible q^n ordered sequences $b_0 b_1 b_2 \cdots b_{n-1}$ over the alphabet $K(|K| = n)$. The cycle of q^n letters is called a q-ary De Bruijn sequence of order n[12–13]. It contains each sub-string of length n exactly once. Then, we will introduce the principle of the cutting-joining cycle method to construct the De Bruijn sequence.

The q^n-node De Bruijn graph is the digraph of which the node-set is denoted by Q, the set of all n-bit strings. Starting from every point, such as $(a_1, a_2, \cdots, a_n)$, there are q directed arcs, of which the destinations are $(a_2, a_3, \cdots, a_n, 0)$, $(a_2, a_3, \cdots, a_n, 1), \cdots, (a_2, a_3, \cdots, a_n, q-1)$, respectively. There are a total of q^{n+1} directed arcs in a De Bruijn graph[14]. A directed graph composed of q^n vertexes and q^{n+1} directed arcs is called the De Bruijn–Good graph of order n, denoted by $\boldsymbol{G}_n$. The state diagram of n-level shift register is a directed graph included in $\boldsymbol{G}_n$. The feedback function $f(x_1, x_2, \cdots, x_n)$ for the shift register is denoted by G_{f}. The number of states in a cycle of G_{f} (length of cycle) is called the period of the cycle, denoted by p. There is a correlation $p \leqslant q^n$ between p and q^n. The sequence with period q^n is called the De Bruijn sequence[12–13].

Suppose the companion of an arbitrary state $A_1 = (a_1, a_2, \cdots, a_n)$ in G_{f} is defined by A', and $B = (b_1, b_2, \cdots, b_n) \in A', (a_i = b_i, i = 1, 2, \cdots, n-1), b_n \in Q, a_n \neq b_n$. The feedback function $f(x_1, x_2, \cdots, x_n) = x_1$ for n-level shift register is a pure rotation of the shift register. Its state graph G_{f} is composed of $l(1 \leqslant l < q^n)$ disjoint circles (length m), where m is the factor of n. For an arbitrary cycle composed of elements of a set, denoted by C, $C = \{A_1, A_2, \cdots, A_m\}$, where $A_i \to A_{i+1}, i = 1, 2, \cdots, m-1, A_m \to A_1, A_i = (a_i, a_{i+1}, \cdots, a_{i+n-1})$, and C can be represented by $C = \{a_1, a_2, \cdots, a_m\}$. State A_1 and its companion state $B \in A'$ are on different cycles. Two cycles C_1 and C_2 are assumed to be adjacent if they are disjoint, and the state A_1 exists on C_1 of which companion B is on C_2. It is well-known that C_1 and C_2 are joined into a single cycle when the predecessors of A_1 and B are interchanged. When all the cycles of a factor are joined into one cycle, this cycle has $m = q^n$ states, and it is called a q-ary De Bruijn sequence of order n[13].

Cutting-joining is an effective method to generate many De Bruijn sequences. If both state A_1 and arbitrary state $B_i(1 \leqslant i \leqslant q-1)$ in a set of $B_1, B_2, \cdots, B_{q-1} \in A'$ belongs to the same cycle C in G_{f}, the predecessors of A_1 and B_i can then be interchanged, and cycle C can be divided into two cycles C_1 and C_2. At the same time, other states in cycle C are stationary, which obtains the state graph G_{f_1} of f_1. This processing is called cutting. If the state A_1 and state B_i belong to two different cycles C_1 and C_2 in G_{f_1}, the predecessors of A_1 in C_1 and B_i in C_2, and cycle C_1 and C_2 can be joined into a single cycle C. At the same time, other states in cycle C are stationary, which obtains the state diagram G_{f_2} of f_1. This processing is called joining. k new De Bruijn sequences can be obtained by one cutting-joining processing from G_{f}, where k is the cross points between two companion states in G_{f}. l new De Bruijn sequences can be obtained by one cutting-joining processing from k new De Bruijn sequences state graph G_{h}, where l is the cross points between two companion states in G_{f}. Until new De Bruijn sequences are generated, the whole q-ary De Bruijn sequences of order n can be obtained.

1.2 Pseudorandom sequence and its construction

Let q be a prime or a power of a prime, and $\mathrm{GF}(q)$ ($\mathrm{GF}(q) = \{0, 1, 2, \cdots, q-1\}$) be the Galois field with q elements. Suppose

$$h(x) = x^n + k_{n-1}x^{n-1} + \cdots + k_1 x + k_0 \tag{1}$$

with $k_i \in \mathrm{GF}(q)$, $k_0 \neq 0$. This polynomial called primitive polynomial $h(x)$ of q elements n degree specifies a feedback shift register. The boxes of the feedback shift register contain elements of $\mathrm{GF}(q)$, say $a_{i+n-1}, a_{i+n-2}, \cdots, a_i$ can be constructed from the shift register. The feedback path then forms

$$a_{i+n} = -k_{n-1}a_{i+n-1} - k_{n-2}a_{i+n-2} - \cdots - k_1 a_{i+1} - k_0 a_i. \tag{2}$$

Equation (2) is the recurrence describing the output sequence. This is an infinite sequence $a_0 a_1 a_2 \cdots$ of period $q^n - 1$ (if the starting state is not zero), and each nonzero state appears once in a period. Any string $a_i a_{i+1} \cdots a_{i+q^n-2}$, $i = 1, 2, \cdots, q^n - 1$ of the output sequence of length $q^n - 1$ is called a pseudorandom

sequence over GF(q) or an M sequence[15]. There are q^n-1 different pseudorandom sequences. The pseudorandom sequence has the window property. If a window of size n is slid along a pseudorandom sequence, every nonzero n-ary is seen exactly once.

2 Construction for bi-dimensional array

It is comparatively easy to use a pseudorandom sequences to obtain an array of dimensions $n_1 \times n_2$, and $N = q^n - 1 = n_1 n_2 = q^{k_1 k_2} - 1$, provided n_1 and n_2 are relatively prime and greater than 1, $n_1 = q^{k_1} - 1$, and $n_2 = \frac{N}{n_1}$. Suppose a pseudorandom sequence $a = a_0 a_1 a_2 \cdots a_{n-1}$ obtained from a primitive polynomial $h(x)$ of degree $n = k_1 k_2$. Sequence a can be used to fill up the array of dimensions $n_1 \times n_2$, by writing a down the main diagonal and continuing from the opposite side whenever an edge is reached. The algorithms of construction of pseudorandom arrays is called the diagonal algorithms[6].

3 New method to construct M-arrays

We describe the proposed piece growing algorithm to construct the M-arrays as follows. The principle is that all sub-windows are treated as pieces. Firstly, all pieces with dimensions $k_1 \times k_2$ of a q-ary n degree M-array are generated, of which any two pieces are not the same. Then, many M-arrays can be constructed by splicing piece by piece.

3.1 Principle of splicing

Suppose that sub-windows (pieces) of dimensions $k_1 \times k_2$ of q-ary n degree M-array among which the dimension of any one is $n_1 \times n_2$, and $k_1 \leqslant n_1, k_2 \leqslant n_2$, denoted by $A|\{A(i,j), 1 \leqslant i \leqslant k_1, 1 \leqslant j \leqslant k_2\}$. The pieces spliced above A are called above-interface pieces, denoted by $A_{\text{above}}|\{A_{\text{above}}(i,j), 0 \leqslant i \leqslant k_1 - 1, 1 \leqslant j \leqslant k_2\}$. In addition, the pieces spliced below A called below-interface are $A_{\text{below}}|\{A_{\text{below}}(i,j), 2 \leqslant i \leqslant k_1 + 1, 1 \leqslant j \leqslant k_2\}$, left A called left-interface are $A_{\text{left}}|\{A_{\text{left}}(i,j), 1 \leqslant i \leqslant k_1, 0 \leqslant j \leqslant k_2 - 1\}$, and right A called right-interface are $A_{\text{right}}|\{A_{\text{right}}(i,j), 1 \leqslant i \leqslant k_1, 2 \leqslant j \leqslant k_2 + 1\}$. Suppose $V = f(A)$, where f is a function of one-to-one correspondence, and V are the values of the piece interface. Hence, the values of up-interface are $V_{\text{above}} = f(A_{\text{above}})$, below-interface $V_{\text{below}} = f(A_{\text{below}})$, left-interface $V_{\text{left}} = f(A_{\text{left}})$, and right-interface $V_{\text{right}} = f(A_{\text{right}})$.

Above-interface and below-interface are called vertical interfaces. Left-interface and right-interface are called horizontal interfaces. The number of vertical interfaces is $NI_{\text{V}} = q^{k_2(k_1-1)}$, and horizontal interfaces $NI_{\text{H}} = q^{(k_2-1)k_1}$.

The set composed of all of the pieces is defined by U. The set composed of the pieces having been utilized to splice is defined by U_{P}. The set composed of unused pieces is denoted by U_{V}. For some reasons, the set composed of useless pieces which can not be spliced into the array is expressed by U_{A}. The correlations between them are expressed as following:

$$\begin{cases} U = U_{\text{P}} \cup U_{\text{V}} \cup U_{\text{A}}, \\ U_{\text{P}} \cap U_{\text{V}} \cap U_{\text{A}} = \emptyset. \end{cases} \tag{3}$$

The number of elements in the array is denoted by N_{U}, $N_{\text{U}} = q^{k_1 k_2}$. Let pieces $P_i, P_j \in U_{\text{P}}$, $P_k \in U_{\text{V}}$, and if P_k splices into the array, it meet the conditions listed below:

(i) P_k splices into the array from the left of P_i (denoted by $P_k \rightarrow P_i$). It meets $V_{\text{left},k} = V_{\text{right},i}$, as shown in Fig.1(a).

(ii) P_k splices into the array from the above of P_j (denoted by $P_k \downarrow P_i$). It satisfies $V_{\text{above},k} = V_{\text{below},j}$, as shown in Fig.1(b).

(iii) P_k splices into the array from the lower-right of P_i and P_j, simultaneously (denoted by $(P_i, P_j)_{\hookleftarrow} \rfloor P_k$). It satisfies $(P_k \downarrow P_i) \cap (P_k \rightarrow P_j)$, namely $V_{\text{above},k} = V_{\text{below},j}$ and $V_{\text{left},k} = V_{\text{right},i}$, as shown in Fig.1(c).

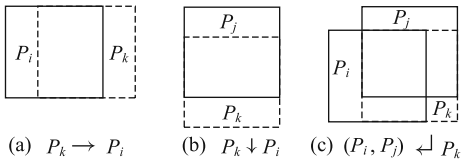


Fig.1 Three splicing conditions

Suppose P_k denotes the successor of the original piece P_i or P_j, namely. When P_k splices into the array, V_{below} and V_{right} will be the exports of original pieces, and V_{above} and V_{left} will be the imports of subsequent pieces, respectively. At the same time, the import value V_{suc} of the successor equals to the export value of the original piece. According to the sub-window property, the number of successors meeting $P_k \rightarrow P_i$ may be q^{k_1}, $P_k \downarrow P_i$ may be q^{k_2}, and $(P_i, P_j)_{\hookleftarrow} \rfloor P_k$ may be q.

Theoretically, the number of successive pieces can get the maximum of pieces. However, it does not mean that it can make maximal successive pieces splice into the array every time. One reason is that part or all of successive pieces satisfying $V_{\text{suc}} = V_{\text{out}}$ are occupied, namely $P_k | V_{\text{suc}} = V_{\text{out}} \notin U_{\text{V}}$. Another is that the import value of original piece equals to their export value, namely $V_{\text{suc},k} = V_{\text{out},k}$. Thus the original piece is an import piece.

3.2 Order of piece splicing

The first piece spliced into the array is called the seed piece, at the upper-left corner of the M-array. Subsequent pieces are chosen to splice into the array based on three methods: $P_k \rightarrow P_i$, $P_k \downarrow P_i$, and $(P_i, P_j)_{\hookleftarrow} \rfloor P_k$, corresponding to the horizontal, vertical, and diagonal

directions, respectively. According to the three methods, there are an export V_{out} originated from the exports of V_{below} or V_{right}, and an import V_{suc} originated from V_{above} or V_{left} to be successors. Figure 2 shows the order of piece splicing into an M-array, where $P_{1,1}$ is the seed piece. The splicing order follows from left to right, from top to bottom, and first the seed piece, then the first row pieces, the first column pieces, the second row pieces, the second column pieces, and so on. Serial number ①, ②, ③, ④, and ⑤ denote the order of the steps to splice into the array. Dashed arrows show the directions of splicing. According to this order, the whole array can be constructed.

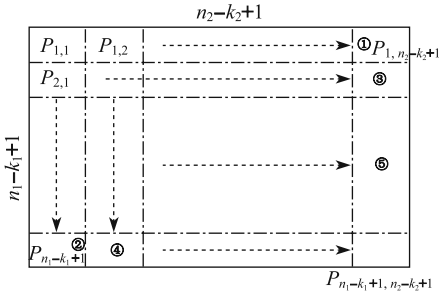


Fig.2 Order of pieces splicing

3.3 Number of M-arrays

The number of a q-ary M-array constructed by using the arbitrary configuration method is $q^{n_1 n_2}$. If we adopt piece growing, the maximum number of pieces will be $N_{\text{U}} = q^{k_1 k_2}$. According to the splicing principle, the number of the chosen seed pieces may be $P_{1,1} = N_{\text{U}}$. The maximum number of $n_2 - k_2$ pieces splicing into the first row is q^{k_1}. The maximum number of $n_1 - k_1$ pieces splicing into the first column is q^{k_1}. The maximum number of $(n_2 - k_2)(n_1 - k_1)$ pieces splicing into other situation is q. The total of M-arrays is $q^{n_1 n_2}$. Namely, the maximum possible number of M-arrays based on the piece growing algorithm is the same as the arbitrary configuration method.

However, the maximum number of successors for the piece growing algorithm can not always be selected. Once a piece is selected, the piece will not appear again. If the selected piece has already appeared in the array, the selected piece will be abandoned. Therefore, the actual number is far less than the maximum number of chosen.

The possible maximum number of seed piece $P_{1,1}$ is N_{U}. The possible maximum number of row 1 column $i(2 \leqslant i \leqslant n_2 - k_2 + 1)$ is $q^{k_1} \frac{N_{\text{U}} - (i-1)}{N_{\text{U}}}$. The possible maximum number of row j column 1 $(2 \leqslant j \leqslant n_1 - k_1 + 1)$ is $q^{k_2} \frac{N_{\text{U}} - (n_2 - k_2 + j - 1)}{N_{\text{U}}}$. The possible maximum number of row j column $i(2 \leqslant j \leqslant n_1 - k_1 + 1)$ and $(2 \leqslant i \leqslant n_2 - k_2 + 1)$ is $q \frac{N_{\text{U}} - (n_2 - k_2 + 1)(i-1) - (j-1)}{N_{\text{U}}}$. Hence the whole possible maximum number of arrays is calculated as

$$
\begin{aligned}
& N_{\text{U}} q^{k_1} \frac{N_{\text{U}} - 1}{N_{\text{U}}} \cdots q^{k_1} \frac{N_{\text{U}} - (n_2 - k_2)}{N_{\text{U}}} \\
& \cdot q^{k_2} \frac{N_{\text{U}} - (n_2 - k_2 + 1)}{N_{\text{U}}} \cdots \\
& \frac{q^{k_2}(N_{\text{U}} - (n_2 + n_1 - k_2 - k_1))}{N_{\text{U}}} \\
& \cdot \frac{q(N_{\text{U}} - (n_2 + n_1 - k_2 - k_1 + 1))}{N_{\text{U}}} \cdots \\
& \frac{q(N_{\text{U}} - (n_2 - k_2 + 1)(n_1 - k_1 + 1))}{N_{\text{U}}} \\
= & \frac{\prod_{i=1}^{(n_2-k_2+1)(n_1-k_1+1)} (N_{\text{U}} - i + 1) q^{k_1(n_2-k_2)} q^{k_2(n_1-k_1)}}{N_{\text{U}}^{(n_2-k_2+1)(n_1-k_1+1)-1}} \\
& \cdot q^{(n_2-k_2)(n_1-k_1)} \\
= & \frac{\prod_{i=1}^{(n_2-k_2+1)(n_1-k_1+1)} (q^{k_1 k_2} - i + 1)}{q^{k_1 k_2 (n_2-k_2+1)(n_1-k_1+1)}} q^{n_1 n_2}. \qquad (4)
\end{aligned}
$$

The maximum number of M-arrays based on the piece growing algorithm can be reduced by $\frac{\prod_{i=1}^{(n_2-k_2+1)(n_1-k_1+1)} (q^{k_1 k_2} - i + 1)}{q^{k_1 k_2 (n_2-k_2+1)(n_1-k_1+1)}}$ times compared with the arbitrary configuration method. For example, to construct a 10-ary M-array of dimensions 59×79 in which there are sub-windows of dimensions 2×2, and the number of arrays is only 3.603 584e533 times less than the arbitrary configuration method. Every sub-window appears only once in the whole array based on the piece growing algorithm. Due to limited space, Fig. 3 shows a part of 59×79 M-array with 2×2 window property.

3771874219726436296780854569774679901425980816209324260329086743670520982751239
3330788772771564101896282719295346665183874793315867906323548478472107159256698
6317936641932142857827261698559984417490333868256350778037229141659003481722519
8996630345539746415023505996170234538286194011993283915240405764261752887663864
8142767856706067558886085114924731212661575710929108874395363460081357872762144
0709455374263503013739096895097764432131987858963694251074185427306826555993437
0048773681248931395636495341694077440593512237604868119902035989284865891807235
0064954401681380600970357801601487145011908963399706482214334805365247213874556
9885515518463631991300473388846302159954979754770361043238516616878758683810315
2683600244874072061713049315394007626101768662681822495190020528194459627419137
4797814107831168492705669755137261286425585361936715120956094747248841039307325
1321696324342070940998834609290698882563471317846023220644839287011496540823558

Fig.3 Partial array of 59×79 M-array with 2×2 window property

4 Application of M-arrays

4.1 Encoded pattern

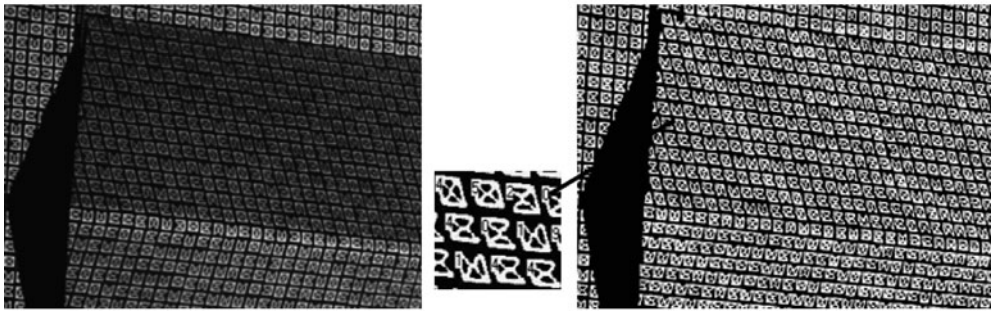
If a 1 024×768 pixels projector plane with code words among which any occupies 11×11 pixels and is spaced by 2 pixels is designed, there will be 79×59 code words on

the plane and 4 661 code words are produced for global codification. If the window property of the array is 2×2, and such a large number of code words are required to generate while utilizing monochrome illumination, at least 10 special symbols should be selected.

In order to test the effectiveness of the encoded pattern for a structured light system, the setup is established, composed of an EPSON EMP-821 LCD projector and a CoolSNAP *cf* CCD camera. It is also applied to implement reconstruction for two objects. The resolutions of the camera and the projector are 1 040×1 392 pixels with pixel size of 4.65 μm×4.65 μm, and 1 024×768 pixels, respectively. Both the camera and the projector are calibrated by Zhang, *et al.*[16] Due to limited space, Fig. 4(a) only shows the partial image for the stairs grabbed by the camera.

4.2 Pattern recognition and decoding

10 symbols in the array have the same features mentioned above. An algorithm called angle variation presented by Yu[17] can be utilized to recognize different symbol. Horns and intersections of symbols are settled in sub-pattern acting as key points and these points have large angle variations. Different symbols have different angle variations, and there are remarkable differences at positive and negative peaks between different symbols. In addition, the number of positive and negative peaks has difference among these 10 symbols. Thus we can robustly identify the 10 symbols. Experiment result is shown in Fig.4(b). The small figure between Figs.4(a) and 4(b) is the enlarged figure from a part of Fig.4(b).



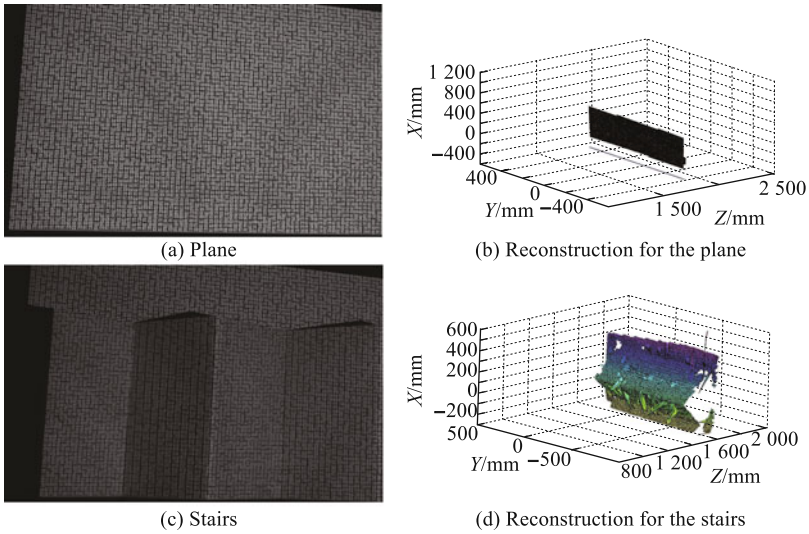
(a) (b)

Fig.4 (a) Partial image grabbed for a stairs and (b) its decoding results

4.3 Three dimensional reconstructions

Using the established experimental platform, the designed pattern is projected onto the object in the scene. The camera grabs the scene image. The process of identification of sub-pattern means the establishment of relation between the distorted sub-pattern and the normal sub-pattern. Using this information, based on the structured light sensing principle, 3D shape of objects can be reconstructed.

Figures 5(b) and 5(d) are the reconstruction results of two objects. There are some cavities and burrs. The reasons are listed below. Firstly, because of uncontinuous scene surface, partial pattern symbols are broken and incomplete, decoding algorithms can not recognize them. Secondly, after symbol image pre-processing, as a result of noise disturbance, partial symbol contours are broken, which disturbs the symbol recognition.



(a) Plane (b) Reconstruction for the plane

(c) Stairs (d) Reconstruction for the stairs

Fig.5 Objects reconstruction

5 Conclusions

This paper has presented an algorithm called piece growing to construct M-arrays. Appropriate pieces (sub-windows) are chosen to splice into the array to construct the whole array. Two steps are vital in the algorithm. First, all sub-windows should be firstly produced, and they are different each other. Second, one sub-window can be spliced into the array only if it does not ever appear in the array. An M-array with dimensions of 79×59 is constructed. A pattern with dimensions of 1 024×768 pixels using 10 symbols based on the M-arrays is proposed with window property 2×2. In addition, the reconstructions of two objects are given. Future work will be focused on the efficiency of construction for M-arrays of n-ary (n >10) and its applications.

References

[1] Salvi J, Fernandez S, Pribanic T, Llado X. A state of the art in structured light patterns for surface profilometry [J]. Pattern Recognition, 2010, 43(8): 2666–2680.

[2] Vargas J, Koninckx T, Quiroga J A, Gool L V. Three-dimensional measurement of microchips using structured light techniques [J]. Optical Engineering, 2008, 47, DOI: 10.1117/1.2919726.

[3] Chen H J, Cheng Y, Liu D D, Zhang X D, Zhang J, Que C L, Wang G F, Fang J. Color structured light system of chest wall motion measurement for respiratory volume evaluation [J]. Journal of Biomedical Optics, 2010, 15, DOI:10.1117/1.3368680.

[4] Salvi J, Pagés J, Batlle J. Pattern codification strategies in structured light systems [J]. Pattern Recognition, 2004, 37(4): 827–849.

[5] Etzion T. Constructions for perfect maps and pseudorandom arrays [J]. IEEE Transaction on Information Theory, 1988, 34(5): 1308–1316.

[6] MacWilliams F J, Sloane N J A. Pseudorandom sequences and arrays [J]. Proceedings of the IEEE, 1976, 64(12): 1715–1729.

[7] Morita H, Yajima K, Sakata S. Reconstruction of surfaces of 3-D objects by M-array pattern projection method [C]// The 2nd International Conference on Computer Vision, Ann Arbor, USA. 1988: 468–473.

[8] Kiyasu S, Hoshino H, Yano K, Fujimura S. Measurement of the 3-D shape of specular polyhedrons using an M-array coded light source [J]. IEEE Instrumentation and Measurement Society, 1995, 44(3): 775–778.

[9] Morano R, Ozturk C, Conn R, Dubin S, Zietz S, Nissano J. Structured light using pseudorandom codes [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1998, 20(3): 322–327.

[10] Petriu E M, Sakr Z, Spoelder H J W, Moica A. Object recognition using pseudorandom color encoded structured light [C]// Proceedings of the 17th IEEE Instrumentation and Measurement Technology Conference, Baltimore, USA. 2000: 1237–1241.

[11] Pages J, Collewet C, Chaumette F, Salvi J, Girona S, Rennes F. An approach to visual servoing based on coded light [C]// Proceedings of 2006 IEEE International Conference on Robotics and Automation, Orlando. 2006: 4118–4123.

[12] Games R A. A Generalized recursive construction for de Bruijn sequences [J]. IEEE Transactions on Information Theory, 1983, 29(6): 843–850.

[13] Annexstein F S. Generating De Bruijn sequences: An efficient implementation [J]. IEEE Computers, 1997, 46(2): 198–200.

[14] Ali A, Halijak C A. The pseudorandom sequence of arrays [J]. Circuits, Systems, and Signal Processing, 1989, 8(2): 133–144.

[15] Ozturk C, Nissanov J, Dubin S. Generation of perfect map codes for an active stereo imaging system [C]// Proceedings of the 22nd IEEE Annual Northeast Bioengineering Conference, New Brunswick. 1996: 76–77.

[16] Zhang Z Y. A flexible new technique for camera calibration [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000, 22(11): 1330–1334.

[17] Yu Qing-cang. A study on symbol M-array based 3D detection approach utilizing binary structured light [D]. Hangzhou: Zhejiang University, 2007 (in Chinese).

(Editor HONG OU)